

HASH SET IN JAVA

DAY 71
19/09/2025

1. What is HashSet?

- A HashSet $\langle E \rangle$ is a collection that stores unique elements only
No Duplicates allowed
- It's a part of Java Collection framework and implements the Set Interface
- It is Backed Internally By a HashMap (just stores elements as keys, with dummy value's)

2. Why HashSet is used?

- When you want to store Unique Values only
- Provides fast lookup, Insertion & deletion (on average $O(1)$ time complexity)
- Useful When order doesn't matter, But uniqueness does

Example: Storing unique roll Numbers, unique usernames, or Unique Words in a text

3. How it Works Internally?

- Under the hood, HashSet Uses a HashMap
- When you add an element $\rightarrow$ hashCode() of the element is calculated
- That hash is used to decide in which Bucket the element will be placed.
- If two elements have the Same hash (collision) $\rightarrow$ It uses Equals() to check Equality
- Only one unique element is kept (no duplicates)

```
HashSet<classDataType> set = new HashSet<>();  
set.add(object);
```

is Equivalent to
`map.put(object, "PRESENT");` // PRESENT is a dummy constant object

✓ 4. Operations on HashSet

Operation	Method	Time Complexity	Description
Add element	<code>add(E e)</code>	$O(1)$ avg	Adds element if not present
Remove element	<code>remove(E e)</code>	$O(1)$ avg	Removes element if present
Check contains	<code>contains(E e)</code>	$O(1)$ avg	Returns true if element exists
Size	<code>size()</code>	$O(1)$	Number of elements
Iterate	<code>iterator()</code> or <code>for-each</code>	$O(n)$	Loops over elements
Clear all	<code>clear()</code>	$O(1)$	Removes all elements
Empty check	<code>isEmpty()</code>	$O(1)$	Checks if empty

java

```
import java.util.*;

public class HashSetDemo {
    public static void main(String[] args) {
        HashSet<String> set = new HashSet<>();

        // Adding elements
        set.add("Apple");
        set.add("Banana");
        set.add("Orange");
        set.add("Apple"); // Duplicate, will be ignored

        // Checking elements
        System.out.println(set.contains("Banana")); // true
        System.out.println(set.contains("Mango")); // false

        // Removing elements
        set.remove("Orange");

        // Iterating
        for (String fruit : set) {
            System.out.println(fruit);
        }

        // Size
        System.out.println("Size: " + set.size());
    }
}
```

Key Points

- No Duplicates
- No guarantee of insertion order
- Backed By Hash Map Internally
- Average $O(1)$ Operation

JAVA Hash Set is directly
Inspired By Mathematical
Sets

◆ Math Set vs. Java HashSet

Concept (Math Set)

Definition: A set is a collection of **distinct elements**.

No Order: In math, sets have no ordering (e.g., $\{1,2,3\} = \{3,2,1\}$).

Membership: You can check if an element belongs to the set.

Union ($A \cup B$): Combine all unique elements from two sets.

Intersection ($A \cap B$): Common elements in both sets.

Difference ($A - B$): Elements in A but not in B.

Subset: $A \subseteq B$ if all elements of A are in B.

Java HashSet Equivalent

HashSet stores **unique elements only** (no duplicates).

HashSet does not guarantee order of elements.

`set.contains(x)` checks if element exists.

`set1.addAll(set2)`

`set1.retainAll(set2)`

`set1.removeAll(set2)`

`set2.containsAll(set1)`

Example Mathematical Sets

$$A = \{1, 2, 3\}$$

$$B = \{3, 4, 5\}$$

A, B are Two sets

$$A \cup B = \{1, 2, 3, 4, 5\}$$

$$A \cap B = \{3\}$$

$$A - B = \{1, 2\}$$

$A \cup B = [1, 2, 3, 4, 5]$
 $A \cap B = [3]$
 $A - B = [1, 2]$

java

```
import java.util.*;

public class SetMathDemo {
    public static void main(String[] args) {
        HashSet<Integer> A = new HashSet<>(Arrays.asList(1, 2, 3));
        HashSet<Integer> B = new HashSet<>(Arrays.asList(3, 4, 5));

        // Union
        HashSet<Integer> union = new HashSet<>(A);
        union.addAll(B);
        System.out.println("A ∪ B = " + union);

        // Intersection
        HashSet<Integer> intersection = new HashSet<>(A);
        intersection.retainAll(B);
        System.out.println("A ∩ B = " + intersection);

        // Difference
        HashSet<Integer> difference = new HashSet<>(A);
        difference.removeAll(B);
        System.out.println("A - B = " + difference);
    }
}
```

So Yes, Hash Set in Java is the programming version of Mathematical Sets - with support for Set theory operations directly via Methods.



Java HashSet — Beginner-Friendly Notes (with diagrams, code & Big-O)

What is it? A `HashSet<E>` stores **unique elements only**. It's **fast on average** for `add`, `contains`, and `remove` — typically $O(1)$. Internally, a `HashSet` uses a `HashMap<E, Object>` under the hood (your elements are used as keys; a dummy value is stored).

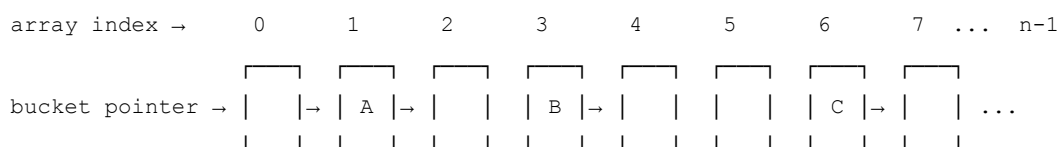
1) Why use a HashSet?

- **No duplicates:** If you add the same element again, it's ignored.
- **Fast membership tests:** “Is x present?” → average $O(1)$.
- **Simple:** You don't need sorting or an order — just uniqueness.

Real-life analogy Think of a **college ID allotment**: each student gets **one unique ID**. If someone tries to register the same ID again, the system rejects it.

2) Quick picture: how it stores things

A `HashSet` piggybacks on `HashMap`'s **array of buckets**. Each element's **hash** decides **which bucket** it lands in.



Legend: A/B/C = linked lists (or trees in Java 8+) of entries that collided in the same bucket

- If two different elements map to the **same bucket**, they form a small **chain** (collision).
- Since Java 8, very long chains may become a **balanced tree** internally so lookups stay healthy.

3) Minimal, very clear example

```

import java.util.HashSet;
import java.util.Set;

public class HashSetBasics {
    public static void main(String[] args) {
        // Create a set of student roll numbers
        Set<Integer> rolls = new HashSet<>();

        // ADD elements (duplicates are ignored)
        rolls.add(101); // true: added
        rolls.add(102); // true: added
        rolls.add(101); // false: already present, ignored

        // CHECK membership
        System.out.println(rolls.contains(102)); // true
        System.out.println(rolls.contains(999)); // false

        // REMOVE an element
        rolls.remove(101); // true: removed

        // SIZE & ITERATION (order is NOT guaranteed)
        System.out.println("Count: " + rolls.size()); // e.g., 1
        for (Integer r : rolls) {
            System.out.println(r);
        }
    }
}

```

What just happened?

- add inserts only if the element is new.
- contains checks fast if present.
- **Order is not guaranteed**; don't rely on iteration order.

4) How does HashSet keep things unique?

1. It computes the element's `hashCode()` to pick a **bucket**.
2. If that bucket has entries, it checks `equals()` to see if an "equal" element is already there.
3. If a **logically equal** element exists → **do not add**; otherwise insert it.

Collision sketch (conceptual):

```

Bucket 5:
[hash=..., key=101] -> [hash=..., key=205] -> null

```

Even if hashes collide, `equals()` ensures we don't add logical duplicates.

5) Time complexity (Big-O)

Operation	Average time	Worst time (Java 8+)	Notes
<code>add(e)</code>	O(1)	$O(\log n) / O(n)^*$	Tree bins keep worst cases near $O(\log n)$
<code>contains(e)</code>	O(1)	$O(\log n) / O(n)^*$	Depends on collisions
<code>remove(e)</code>	O(1)	$O(\log n) / O(n)^*$	Similar reasoning
Iterate elements	O(n)	$O(n)$	Visit each element once

*For beginners: assume **O(1)** average for `add/contains/remove`.*

6) Important facts you'll use a lot

- **No duplicates:** If `equals()` says two elements are equal, the second won't be added.
- **Null:** `HashSet` allows **one** `null` element.
- **No order:** Iteration order is **unspecified** and may change after modifications.
- **Fail-fast:** Modifying the set while iterating (except via iterator's `remove()`) may throw `ConcurrentModificationException`.
- **Thread-safety:** Not thread-safe by default (see concurrency options below).

7) Custom element types (beginner-safe version)

If you store your **own class** inside a `HashSet`, you **must** define equality properly.

```

import java.util.HashSet;
import java.util.Objects;
import java.util.Set;

// We'll consider two Student objects "the same" if they share the same rollNo.
// Keep fields used for equality IMMUTABLE to avoid bugs.
final class Student {
    private final int rollNo;    // used for equality
    private final String name;   // NOT used for equality here

    Student(int rollNo, String name) {
        this.rollNo = rollNo;
        this.name = name;
    }

    // Two Students are equal if rollNo matches
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;           // same object
        if (!(o instanceof Student)) return false;
        Student s = (Student) o;
        return this.rollNo == s.rollNo;       // compare key field(s)
    }

    // Must be consistent with equals: equal rollNo -> equal hash
    @Override
    public int hashCode() {
        return Objects.hash(rollNo);
    }

    @Override
    public String toString() {
        return rollNo + ":" + name;
    }
}

public class CustomTypeInSet {
    public static void main(String[] args) {
        Set<Student> set = new HashSet<>();

        set.add(new Student(101, "Asha"));      // added
        set.add(new Student(101, "Asha Updated")); // NOT added (same rollNo)
        set.add(new Student(102, "Ravi"));      // added

        System.out.println(set.size()); // 2
        System.out.println(set);        // order not guaranteed
    }
}

```

```
}  
  
}
```

Rule of thumb:

- Make fields used in `equals/hashCode` **immutable** (e.g., `final`).
- If you mutate them after insertion, the set may not find your element later.

8) Resizing (what happens when it grows?)

- Backed by a `HashMap`, the set's table has a **capacity** and a **load factor** (default ~0.75).
- When it gets "too full", it **resizes** (usually doubles capacity) and **re-distributes** elements to keep buckets short.

If you know the size in advance, pre-size to avoid multiple resizes:

```
int expected = 10_000;  
float load = 0.75f;  
// initialCapacity ≈ expected / load  
Set<Integer> ids = new HashSet<>((int) Math.ceil(expected / load), load);
```

9) Common beginner patterns

A) Remove duplicates from a list (order does NOT matter)

```
import java.util.*;  
  
public class DedupQuick {  
    public static void main(String[] args) {  
        List<String> names = List.of("Asha", "Asha", "Ravi", "Meera", "Ravi");  
        Set<String> unique = new HashSet<>(names); // constructor removes duplicates  
        System.out.println(unique); // order not guaranteed  
    }  
}
```

B) Fast membership filter (e.g., blocked emails)

```

import java.util.*;

public class BlockList {
    public static void main(String[] args) {
        Set<String> blocked = new HashSet<>();
        blocked.add("spam@x.com");
        blocked.add("bot@y.com");

        String incoming = "bot@y.com";
        if (blocked.contains(incoming)) {
            System.out.println("Reject mail");
        } else {
            System.out.println("Accept mail");
        }
    }
}

```

C) Set operations: union / intersection / difference

```

import java.util.*;

public class SetOps {
    public static void main(String[] args) {
        Set<String> a = new HashSet<>(Set.of("apple", "banana", "mango"));
        Set<String> b = new HashSet<>(Set.of("banana", "cherry"));

        // UNION: elements in a OR b
        Set<String> union = new HashSet<>(a);
        union.addAll(b); // O(|a| + |b|) average
        System.out.println(union); // [apple, banana, mango, cherry]

        // INTERSECTION: elements in BOTH a & b
        Set<String> inter = new HashSet<>(a);
        inter.retainAll(b); // O(min(|a|, |b|)) average
        System.out.println(inter); // [banana]

        // DIFFERENCE: elements in a but NOT in b
        Set<String> diff = new HashSet<>(a);
        diff.removeAll(b); // O(|a| + |b|) average
        System.out.println(diff); // [apple, mango]
    }
}

```

Big-O intuition: these bulk ops iterate through elements and do $O(1)$ average set checks.

10) Concurrency (just enough for college use)

- `HashSet` is **not** thread-safe. For simple synchronization:

```
Set<Integer> syncSet = java.util.Collections.synchronizedSet(new HashSet<>());
```

- For highly concurrent writes/reads, prefer:

```
// Backed by ConcurrentHashMap, good for concurrent membership checks
Set<String> concurrent = java.util.concurrent.ConcurrentHashMap.newKeySet();
```

11) When **not** to use `HashSet`

- If you need **insertion order**, use `LinkedHashSet`.
- If you need **sorted** elements, use `TreeSet` ($O(\log n)$ operations).
- If you need **counts** or key→value mapping, use `HashMap`.

12) Quick checklist (for exams & interviews)

- **Purpose:** store **unique** elements, **fast** membership tests.
- **Average time:** `add/contains/remove` → $O(1)$.
- **One** `null` allowed.
- **No guaranteed order.**
- For custom types: **override** `equals` and `hashCode`; keep those fields **immutable**.
- Pre-size for big datasets to reduce resizing.

TL;DR

- `HashSet` = **unique bag** with **fast checks**.
- Visualize “**index** → **bucket** → **tiny chain/tree**”.
- Use it whenever you need **no duplicates** and **quick membership** — not ordering or sorting.